Formalizing Analytic Number Theory in Lean 4: Practical Lessons from the Cathedral Project

Jason Robert Gochanour

April 20, 2026

Abstract

We describe the practical techniques for formalizing analytic number theory in Lean 4 with Mathlib, distilled from the Cathedral project: a 22,670-line formalization of the Nyman–Beurling criterion for the Riemann Hypothesis. We cover Mathlib API patterns that worked well (interval integrals, spectral theory), patterns that required workarounds (infinite sums, measure theory), and patterns that are currently missing (analytic continuation, contour integrals). We provide reusable proof idioms for common tasks: bounding L^2 norms, establishing positive definiteness, controlling Dirichlet series, and managing axiom hygiene in evolving codebases. We document 12 specific Mathlib lemmas that were essential, 5 API gaps that required axioms, and 3 deprecated-API traps. This paper is intended as a practical guide for anyone formalizing analytic number theory in Lean 4.

Contents

1 Introduction: Why Analytic Number Theory Is Hard to Formalize **2**
1.1 Scope . . . 2

2 Mathlib APIs That Worked Well **2**
2.1 Interval Integrals . . . 2
2.2 Matrix API . . . 3
2.3 Spectral Theory . . . 3
2.4 Convergence and Limits . . . 3

3 Mathlib APIs That Required Workarounds **4**
3.1 Infinite Series and Dirichlet Series . . . 4
3.2 Fractional Part . . . 4
3.3 Measure Theory Coercions . . . 4

4 Mathlib Gaps That Required Axioms **5**
4.1 The Analytic Continuation Gap . . . 5
4.2 The Contour Integral Gap . . . 5

5 Twelve Essential Mathlib Lemmas **5**

6 Project Structure Advice **6**
6.1 The Layered Module Pattern . . . 6
6.2 The Axiom Registry Pattern . . . 6
6.3 The Sorry Hygiene Pattern . . . 7

7 Common Tactic Patterns **7**
7.1 The linarith-norm_num One-Two Punch 7
7.2 The ring-then-bound Pattern 7
7.3 The convert-congr Escape Hatch 8

8 Deprecated API Traps **8**

9 Verification Checklist **8**

10 Recommended Mathlib Contributions **9**

11 Conclusion **9**

1 Introduction: Why Analytic Number Theory Is Hard to Formalize

Analytic number theory sits at the intersection of analysis, algebra, and combinatorics. A typical proof might use:

- Complex analysis (contour integrals, residue calculus).
- Real analysis (L^2 norms, Parseval's theorem).
- Combinatorics (Möbius inversion, sieve methods).
- Linear algebra (matrix eigenvalues, Schur complements).

Each of these is well-developed in Mathlib individually. The challenge is using them *together*, in a single proof, with consistent type coercions and compatible assumptions.

The Cathedral project encountered this challenge over 23 days, producing 84 active Lean files. This paper distills the lessons into actionable advice for future formalizers.

1.1 Scope

This paper is about *practice*, not theory. We assume the reader is familiar with Lean 4 syntax and has basic Mathlib experience. We do not explain the mathematics of the Nyman–Beurling criterion (see the companion paper [3]). Instead, we focus on:

- Which Mathlib APIs to use (and which to avoid).
- How to structure a large proof project.
- How to manage axioms in evolving codebases.
- How to avoid common traps.

2 Mathlib APIs That Worked Well

2.1 Interval Integrals

Mathlib's `MeasureTheory.integral` and `intervalIntegral` work well for L^2 norms on $(0, 1)$:

Listing 1: L2 norm on (0,1)

```
-- The BD distance as an interval integral
noncomputable def nbDistSq (N : Nat) (v : Fin (N-1) -> Real) : Real :=
  integral_Ioo 0 1
    (fun x => (1 - bdLinComb N v x) ^ 2)
```

Key lemma: `MeasureTheory.integral_nonneg` is essential for establishing $d_N^2 \geq 0$. Use it with `sq_nonneg` for squaredness.

Pattern: For bounding $\int_a^b f(x)^2\,dx$, use `MeasureTheory.integral_mono_of_nonneg` with `sq_le_sq'` or `abs_le`.

2.2 Matrix API

Mathlib's matrix library is excellent for Gram matrices:

Listing 2: Positive definiteness

```
-- The Gram matrix is positive definite
theorem gram_posDef (N : Nat) (hN : 2 <= N) :
    Matrix.PosDef (gramMatrix N) := by
  intro v hv
  -- Key: express v^T G v as an L2 norm
  rw [gram_quadratic_form_eq_integral]
  exact integral_sq_pos_of_ne_zero v hv
```

Key lemma: `Matrix.PosDef.det_pos` connects positive definiteness to invertibility. Essential for the Rayleigh quotient.

Key lemma: `Matrix.nonsing_inv` provides the inverse, but requires a `Fintype` instance and `DecidableEq` on the index type.

2.3 Spectral Theory

For eigenvalue bounds, use the Rayleigh quotient approach:

Listing 3: Rayleigh quotient bound

```
-- All eigenvalues are bounded below
theorem eigenvalue_lower_bound
    (A : Matrix (Fin n) (Fin n) Real) (hA : A.IsHermitian) :
    forall i, hA.eigenvalues i >= lambda_min := by
  intro i
  exact hA.eigenvalues_ge_of_forall_rayleigh_le ...
```

Pattern: For symmetric real matrices, use `Matrix.IsHermitian` (which handles the real case via `starRingEnd`). The eigenvalue API is on `IsHermitian`, not on a separate "symmetric" type.

Trap: The API `Matrix.isHermitian_iff_isSymmetric` was deprecated in recent Mathlib, but no replacement exists. We kept the deprecated call with a comment.

2.4 Convergence and Limits

For $\ln(\ln N)/\ln N \to 0$:

Listing 4: Algebraic limit proof

```
-- Avoid Filter.Tendsto arguments entirely
-- Use the algebraic bound: log x <= 2 * sqrt x
theorem log_le_two_sqrt (x : Real) (hx : 0 < x) :
    Real.log x <= 2 * Real.sqrt x := by
```

```
  ...

-- Then bound log(log N) / log N <= 2 * sqrt(log N) / log N
-- which goes to 0 because sqrt(x)/x = 1/sqrt(x) -> 0
```

Pattern: When possible, avoid `Filter.Tendsto` and `Filter.atTop` entirely. Algebraic bounds with explicit inequalities are much easier to formalize and audit.

3 Mathlib APIs That Required Workarounds

3.1 Infinite Series and Dirichlet Series

Mathlib's `tsum` (topological infinite sums) requires `Summable` hypotheses that are often difficult to establish for Dirichlet series. The `hasSum` API is more flexible but verbose.

Workaround: We frequently used truncated finite sums (`Finset.sum`) with explicit error bounds, rather than working with infinite sums directly. This adds lines but avoids the `Summable` burden.

3.2 Fractional Part

There is no `Int.fract` for real-valued fractional parts on $(0, 1)$ with the properties we need. We defined our own:

Listing 5: Fractional part definition

```
noncomputable def fracPart (x : Real) : Real :=
  x - Real.floor x

-- Key property we proved manually:
theorem fracPart_mem_Ioo (x : Real) (hx : 0 < x)
    (hn : not (exists n : Int, x = n)) :
    fracPart x in Set.Ioo 0 1 := by ...
```

Note: Mathlib's `Int.fract` exists but its API is sparse for the properties needed in analytic number theory (integrability, measurability, interaction with floor sums). Expect to prove many basic properties yourself.

3.3 Measure Theory Coercions

Moving between `MeasureTheory.integral` (Bochner integral) and `intervalIntegral` (Henstock–Kurzweil) requires care:

- `MeasureTheory.integral` works with `MeasureTheory.Integrable`.
- `intervalIntegral` works with `IntervalIntegrable`.
- The connection is via `intervalIntegrable_iff_integrable_Ioc`.

Trap: The two integrals may disagree on sets of measure zero. For continuous functions on compact intervals, they agree. For discontinuous functions (like the fractional part), use the Bochner integral with explicit `Integrable` proofs.

4 Mathlib Gaps That Required Axioms

Five Mathlib gaps required axioms in the Cathedral:

Gap
Analytic continuation
Contour integrals
Mertens function
Montgomery–Vaughan
Dedekind sums & Vasyunin cotangent formula via reciprocity (diagonal case proved; off-diagonal requires Gau

4.1 The Analytic Continuation Gap

The most impactful gap. The Mellin transform identity

$$\int_0^1 \{1/(kx)\}\, x^{s-1}\, dx = \frac{k^{-s}}{s(s-1)}\big(s\zeta(s+1)/k - (s-1)\zeta(s)/k + 1\big)$$

is proved for $\operatorname{Re} s > 1$ via direct computation (floor sums). Extending to $\operatorname{Re} s > 0$ requires analytic continuation, which Mathlib supports in principle via `AnalyticAt` and `HasFPowerSeriesAt`, but the infrastructure for Mellin-type integrals is not yet developed.

Recommended approach: Prove the integral converges absolutely for $\operatorname{Re} s > 0$ (it does, since $|x^{s-1}| = x^{\sigma-1}$ is integrable on $(0,1)$ for $\sigma > 0$), then show the right-hand side is meromorphic (it is, being a rational function of $\zeta(s)$ and k^{-s}), and apply the identity theorem.

4.2 The Contour Integral Gap

Mathlib has no built-in support for complex line integrals of the form $\int_{1/2-i\infty}^{1/2+i\infty} f(s)\, ds$. This is the most significant gap for analytic number theory formalization.

Recommended approach: Parameterize as $\int_{-\infty}^{\infty} f(1/2+it)\, i\, dt$ and use real-variable integration after splitting into real and imaginary parts.

5 Twelve Essential Mathlib Lemmas

The following Mathlib lemmas were used repeatedly throughout the Cathedral. Any analytic number theory formalization will likely need them:

#	Lemma	Use
1	`integral_nonneg`	$\int f \geq 0$ when $f \geq 0$
2	`integral_mono_of_nonneg`	Bounding integrals from above
3	`sq_nonneg`	$x^2 \geq 0$ (used constantly)
4	`Matrix.PosDef.det_pos`	Gram matrix invertibility
5	`Matrix.nonsing_inv`	Matrix inversion
6	`Real.log_le_sub_one_of_le`	Log inequalities
7	`Real.rpow_natCast`	(x^n) coercion
8	`Finset.sum_le_sum`	Bounding finite sums
9	`riemannZeta_ne_zero_of_one_le_re`	$\zeta(s) \neq 0$ for $\mathrm{Re}\, s \geq 1$
10	`integral_univ_inv_one_add_sq`	$\int_{-\infty}^{\infty}(1+t^2)^{-1}\,dt = \pi$
11	`MonotoneBounded.tendsto`	Bounded monotone convergence
12	`Nat.Coprime.gcd_eq_one`	GCD properties for Gram entries

Lemma 9 (`riemannZeta_ne_zero_of_one_le_re`) is particularly valuable: it gives $\zeta(s) \neq 0$ for $\mathrm{Re}\, s \geq 1$ for free, which is the classical non-vanishing result essential for the converse direction.

6 Project Structure Advice

6.1 The Layered Module Pattern

We recommend organizing large formalizations into layers:

1. **Layer 0: Definitions**. Core types and definitions. No theorems, no axioms. Everything imports this.
2. **Layer 1: Foundations**. Independent mathematical modules (linear algebra, measure theory). No cross-dependencies.
3. **Layer 2: Theory**. Domain-specific modules that may depend on Layer 1 but not on each other.
4. **Layer 3: Bridges**. Modules that connect Layer 2 modules (e.g., Parseval Bridge connecting L^2 norms to frequency domain).
5. **Layer 4: Assembly**. The crown theorem, importing from all layers.

This structure ensures that the dependency graph is a DAG with bounded depth. It also enables parallel development: two people (or human-AI pairs) can work on different Layer 2 modules without interference.

6.2 The Axiom Registry Pattern

Maintain a central file (`Axioms.lean`) that documents every axiom, even if axioms are declared in their respective modules:

Listing 6: Axiom registry pattern

```
/-!
# Cathedral Axiom Registry
Total: 42 axioms (7 on crown path)
Last audit: April 20, 2026

## Crown Path Axioms (compiler-verified)
1. rh_implies_mertens_bound            -- RH content
```

```
2. pnt_mu_div_k                       -- PNT limit
3. pnt_mu_log_div_k                   -- PNT limit
4. pnt_mu_log_sq_div_k                -- PNT limit
5. abel_mertens_tail_raw              -- Abel summation
6. millennium_covariance_cancellation -- Covariance
7. vasyunin_offdiag_integral          -- Off-diag Gram

## Eliminated Axioms (April 20, 2026)
-- vasyunin_eq_integral  -> THEOREM (diagonal proved)
-- fract_sq_integral     -> THEOREM (Stirling+Squeeze)
-- rh_implies_mertens_34 -> THEOREM (absorbed by _bound)
...
-/
```

Run `#print axioms` on the crown theorem after every structural change. Discrepancies between the registry and the output are bugs.

6.3 The Sorry Hygiene Pattern

1. **Never `sorry` on the critical path.** If the crown theorem depends on a `sorry`, the entire formalization is untrustworthy. Route `sorry` to alternative paths only.
2. **Document every `sorry`** with a comment explaining what it assumes and why it can't be proved yet.
3. **Track `sorry` count** as a project metric. The Cathedral maintains: 0 on crown path, 2 on alternative paths.

7 Common Tactic Patterns

7.1 The linarith-norm_num One-Two Punch

For numerical inequalities ($2 \leq N$, $0 < 1/k$), the combination of `linarith` and `norm_num` handles almost everything:

Listing 7: Numerical inequality dispatch

```
example (N : Nat) (hN : 2 <= N) (k : Fin (N-1)) :
    (0 : Real) < 1 / (k.val + 2 : Real) := by
  positivity  -- or: field_simp; positivity
```

7.2 The ring-then-bound Pattern

For algebraic identities followed by analysis:

Listing 8: Algebra then analysis

```
-- Prove: (1 - x)^2 = 1 - 2x + x^2
-- Then bound: 1 - 2x + x^2 <= 1 when 0 <= x <= 1
calc (1 - x)^2
    = 1 - 2*x + x^2 := by ring
  _ <= 1 := by nlinarith [sq_nonneg x, sq_abs x]
```

7.3 The convert-congr Escape Hatch

When Lean's type unifier fails on goals that are "obviously" equal:

Listing 9: Type unification escape

```
-- Goal: integral f = integral g
-- When f and g are definitionally equal but Lean can't see it
convert this_integral_eq using 1
congr 1
ext x
simp [fracPart, bdBasis]
```

Warning: Overusing `convert` makes proofs fragile. Use it as a last resort, not a first instinct.

8 Deprecated API Traps

Mathlib evolves rapidly. Three deprecated APIs caused problems:

Deprecated	Replacement	Status
`isHermitian_iff_isSymmetric`	(none yet)	Kept with comment
`Integrable.bdd_mul'`	`Integrable.bdd_mul`	Updated
`measurable_of_continuousOn`	`Continuous.measurable`	Updated

Advice: When Lean warns "deprecated," check whether the replacement actually exists in your Mathlib version. Sometimes the deprecation is added before the replacement is ready.

9 Verification Checklist

Before declaring a formalization "done," verify:

- ☐ `lake build` passes with zero errors.
- ☐ `#print axioms crown_theorem` shows only expected axioms.
- ☐ No `sorry` on the crown path.
- ☐ All `sorry` markers documented.
- ☐ The lakefile matches the filesystem exactly.
- ☐ All docstrings reference current axiom counts and file names.
- ☐ Ghost axiom detection script passes (no axiom/theorem duplicates).
- ☐ Numerical validation agrees with formal claims (if applicable).
- ☐ All deprecated API calls documented.

10 Recommended Mathlib Contributions

Based on our experience, the following Mathlib additions would have the highest impact for analytic number theory formalization:

1. **Vertical contour integrals**: A definition and basic API for $\int_{\sigma-i\infty}^{\sigma+i\infty} f(s)\,ds$, parameterized as $\int_{-\infty}^{\infty} f(\sigma+it)\,dt$.
2. **Analytic continuation for Mellin transforms**: Given $F(s) = \int_0^\infty f(x)x^{s-1}dx$ converges for $\operatorname{Re} s \in (a,b)$, show F is analytic on the strip.
3. **Mertens function**: Formalize $M(x) = \sum_{n\le x} \mu(n)$ with basic bounds ($M(x) = O(\sqrt{x}\log x)$ unconditionally).
4. **Fractional part API enrichment**: Integrability, measurability, and L^2 properties of $\{f(x)\}$.
5. **Mean value theorems for Dirichlet polynomials**: Montgomery–Vaughan in its simplest form.

These five additions would eliminate 80% of the axioms in the Cathedral and enable substantially cleaner formalizations of analytic number theory.

11 Conclusion

Formalizing analytic number theory in Lean 4 is feasible today, but requires significant infrastructure work beyond what Mathlib currently provides. The main gaps are in complex analysis (contour integrals, analytic continuation) and specialized number theory (Mertens function, Dirichlet polynomial bounds).

The practical advice in this paper—which APIs to use, how to structure projects, how to manage axioms—should help future formalizers avoid the 96 dead-end files we had to archive.

The Cathedral took 23 days and produced 22,670 lines of active code. The next analytic number theory formalization, building on these lessons, should be faster. The tools are ready. The patterns are documented. Build your cathedral.

References

[1] L. de Moura et al., *The Lean 4 Theorem Prover*, CADE-28, 2021.

[2] The Mathlib Community, *Mathlib4*, https://github.com/leanprover-community/mathlib4.

[3] J.R. Gochanour, *The Cathedral: A Machine-Verified Axiomatic Reduction of the Riemann Hypothesis via the Nyman–Beurling Criterion*, 2026.

[4] The Mathlib Community, *Mathlib Tactics Reference*, https://leanprover-community.github.io/mathlib4_docs/.